

PROGRAMAÇÃO DE JOGOS: SOLUÇÃO DE SUDOKU



D.1 Introdução

Em 2005, o jogo de Sudoku tornou-se popular no mundo inteiro. Agora, quase todo jornal importante publica um quebra-cabeça de Sudoku diariamente. Os jogadores de jogos portáteis permitem que você jogue a qualquer hora, em qualquer lugar, e criam quebra-cabeças por demanda, com vários níveis de dificuldade.

Um **quebra-cabeça Sudoku** é uma grade de 9×9 (ou seja, um array bidimensional) em que os dígitos de 1 a 9 aparecem uma única vez em cada linha, em cada coluna e em cada uma das nove grades de 3×3 . Na grade parcialmente completada de 9×9 da Figura D.1, a linha 1, a coluna 1 e a grade de 3×3 no canto superior esquerdo do tabuleiro contêm os dígitos de 1 a 9 uma única vez. Usamos as convenções de numeração de linha e coluna de array bidimensional de C, mas estamos ignorando a linha 0 e a coluna 0, em conformidade com as convenções da comunidade do Sudoku.

O Sudoku típico oferece muitas células preenchidas e muitos espaços, normalmente arrumados em um padrão simétrico, como é comum em palavras cruzadas. A tarefa do jogador é preencher os espaços para completar o quebra-cabeça. Alguns são fáceis de resolver; alguns são muito difíceis, exigindo estratégias de solução sofisticadas.

	1	2	3	4	5	6	7	8	9
1	5	1	3	4	9	7	6	2	8
2	4	6	8						
3	7	9	2						
4	2								
5	9								
6	3								
7	8								
8	1								
9	6								

Figura D.1 ■ Grade de 9×9 do Sudoku parcialmente completada. Observe as nove grades de 3×3 .

Discutiremos diversas estratégias de solução simples e sugeriremos o que fazer quando estas falharem. Também apresentaremos técnicas para programar criadores e solucionadores de Sudoku em C. Infelizmente, o padrão em C não inclui capacidades gráficas e de GUI (graphical user interface, interface gráfica do usuário), de modo que nossa representação do tabuleiro não será tão elegante quanto poderíamos torná-la em Java e em outras linguagens de programação que admitem essas capacidades. Você pode querer retornar aos programas de Sudoku depois de ler o Apêndice E: programação de jogos usando a biblioteca de C Allegro. Allegro, que não faz parte do padrão C, oferece capacidades que o ajudarão a acrescentar gráficos e até mesmo sons em seus programas de Sudoku.

D.2 Deitel Sudoku Resource Center

Verifique nosso **Sudoku Resource Center** em www.deitel.com/sudoku. Ele contém downloads, tutoriais, livros, e-books e outros itens que o ajudarão a dominar o jogo. Acompanhe a história do Sudoku desde a sua origem no século oitavo até os tempos modernos. Faça o download gratuito do Sudoku em vários níveis de dificuldade, entre em disputas de jogos diárias para ganhar livros de Sudoku e receba diariamente um jogo de Sudoku para postar em sua página na web. Adquira excelentes recursos para iniciantes — aprenda as regras do Sudoku, receba dicas para solucionar amostras dos quebra-cabeças, aprenda as melhores estratégias de solução e adquirir solucionadores de Sudoku gratuitos — basta digitar o quebra-cabeça de seu jornal ou site de Sudoku favorito e adquirir uma solução imediata; alguns solucionadores do Sudoku oferecem até mesmo explicações detalhadas, passo a passo. Adquira jogos de Sudoku para dispositivos móveis, que podem ser instalados em telefones celulares, dispositivos Palm®, players Game Boy® e dispositivos habilitados com Java. Alguns sites de Sudoku possuem temporizadores, sinalizam quando um número incorreto é inserido na grade e oferecem dicas. Compre camisetas e canecas estampadas com Sudoku, participe de fóruns de jogadores, adquira planilhas de Sudoku vazias, que podem ser impressas, e adquira jogos portáteis — um deles oferece um milhão de quebra-cabeças em cinco níveis de dificuldade. Faça o download gratuito do software criador de Sudoku. E, para os que não têm problemas de coração, experimente resolver Sudokus especialmente difíceis com reviravoltas intrincadas, um Sudoku circular e uma variante do quebra-cabeça com cinco grades entrelaçadas. Assine nosso boletim gratuito, o *Deitel® Buzz Online*, para receber notificações das atualizações de nosso Sudoku Resource Center e de outros Deitel Resource Centers em www.deitel.com, que oferecem jogos, quebra-cabeças e outros projetos de programação interessantes.

D.3 Estratégias de solução

Chamamos a uma grade de 9×9 de Sudoku de array s . Examinando todas as células preenchidas na linha, coluna e grade de 3×3 que inclui uma célula vazia em particular, o valor para essa célula pode se tornar óbvio. Logicamente, a célula $s[1][7]$ na Figura D.2 precisa ser 6.

	1	2	3	4	5	6	7	8	9
1	2	9	3	1	8	7	—	5	4
2									
3									
4									
5									
6									
7									
8									
9									

Figura D.2 ■ Determinando o valor de uma célula após a verificação de todas as células preenchidas na mesma linha.

De forma menos lógica, para determinar o valor de $s[1][7]$ na Figura D.3, você precisa obter dicas da linha 1 (ou seja, os dígitos 3, 6 e 9 já foram usados), coluna 7 (ou seja, os dígitos 4, 7 e 1 já foram usados) e da grade superior direita de 3×3 (ou seja, os dígitos 9, 8, 4 e 2 já foram usados). Aqui, a célula vazia $s[1][7]$ *precisa* ser 5 — o único número que ainda não apareceu na linha 1, na coluna 7 ou na grade de 3×3 superior direita.

	1	2	3	4	5	6	7	8	9
1			3			6	—		9
2								8	
3							4		2
4									
5									
6							7		
7							1		
8									
9									

Figura D.3 ■ Determinando o valor de uma célula após verificação de todas as células preenchidas na mesma linha, coluna e grade de 3×3 .

Singletons

As estratégias discutidas até aqui podem facilmente determinar os dígitos finais de algumas células abertas, mas você normalmente terá de se empenhar profundamente. A coluna 6 da Figura D.4 mostra células com valores já determinados (por exemplo, $s[1][6]$ é um 9, $s[3][6]$ é um 6 etc.) e as células que indicam o conjunto de valores (que chamamos de ‘possíveis’) que, nesse momento, ainda são possíveis para essa célula.

	1	2	3	4	5	6	7	8	9
1						9			
2						2 7			
3						6			
4						4			
5						2 7			
6						2 5 7			
7						8			
8						3			
9						1			

Figura D.4 ■ Notação que mostra os conjuntos completos de valores possíveis para células abertas.

A célula $s[6][6]$ contém 257, indicando que somente os valores 2, 5 ou 7 poderão, eventualmente, ser atribuídos a essa célula. As outras duas células abertas na coluna 6 — $s[2][6]$ e $s[5][6]$ — são ambas 27, indicando que *somente* os valores 2 ou 7 poderão, eventualmente, ser atribuídos a essas células. Assim, $s[6][6]$, a única célula na coluna 6 que lista 5 como valor possível restante, *precisa* ser 5. Chamamos esse valor 5 de **singleton**. Assim, podemos dar à célula $s[6][6]$ um 5 (Figura D.5), simplificando um pouco o quebra-cabeça.

	1	2	3	4	5	6	7	8	9
1						9			
2						2 7			
3						6			
4						4			
5						2 7			
6						5			
7						8			
8						3			
9						1			

Figura D.5 ■ Dando à célula $s[6][6]$ o valor singleton 5.

Duplas

Considere a grade de 3×3 superior direita na Figura D.6. As células tracejadas já poderiam ser confirmadas ou poderiam ter listas de valores possíveis. Observe as **duplas** — as duas células $s[1][9]$ e $s[2][7]$ que contêm apenas as duas possibilidades 1 e 5. Se $s[1][9]$ se tornar 1, por fim, então $s[2][7]$ *precisa* ser 5; se $s[1][9]$ se tornar 5, por fim, então $s[2][7]$ *precisa* ser 1. Assim, entre eles, essas duas células definitivamente ‘gastarão’ o 1 e o 5. Assim, o 1 e o 5 podem ser eliminados da célula $s[3][9]$ que contém os valores possíveis 1357, de modo que podemos reescrever seu conteúdo como 37, simplificando um pouco mais o quebra-cabeça. Se, originalmente, a célula $s[3][9]$ tivesse contido apenas 135, então eliminar o 1 e o 5 nos permitiria forçar a célula para o valor 3.

	1	2	3	4	5	6	7	8	9
1							—	—	1 5
2							1 5	—	—
3							—	—	1 3 5 7
4									
5									
6									
7									
8									
9									

Figura D.6 ■ Uso de duplas para simplificar um quebra-cabeça.

Duplas podem ser mais sutis. Por exemplo, suponha que duas células de uma linha, coluna ou grade de 3×3 tenham listas possíveis de 2467 e 257, e que nenhuma outra célula nessa linha, coluna ou grade de 3×3 mencione 2 ou 7 como um valor possível. Então, 27 é uma **dupla oculta** — uma daquelas duas células precisa ser 2, e a outra precisa ser 7, de modo que todos os dígitos diferentes de 2 e 7 podem ser removidos das listas possíveis dessas duas células (ou seja, 2467 torna-se 27 e 257 torna-se 27 — criando um par de duplas — simplificando um pouco o quebra-cabeça).

Triplas

Considere a coluna 5 da Figura D.7. As células tracejadas já poderiam estar confirmadas ou poderiam ter listas de valores possíveis. Observe as **triplas** — as três células contendo exatamente as mesmas três possibilidades 467, a saber, as células $s[1][5]$, $s[6][5]$ e $s[9][5]$. Se uma dessas três células finalmente se tornar 4, então as outras serão reduzidas a duplas de 67; se uma dessas três células finalmente se tornar 6, então as outras serão reduzidas a duplas de 47; se, por fim, uma dessas três células se tornar 7, então as outras serão reduzidas a duplas de 46. Entre as três células contendo 467, uma precisa ser 4, uma precisa ser 6 e uma precisa ser 7. Assim, o 4, 6 e 7 podem ser eliminados da célula $s[4][5]$ que contém os possíveis 14567, de modo que podemos reescrever seu conteúdo como 15, simplificando um pouco o quebra-cabeça. Se a célula $s[4][5]$ tivesse originalmente 1467, então a eliminação do 4, do 6 e do 7 nos permitiria forçar o valor 1 nessa célula.

As triplas podem ser mais sutis. Suponha que uma linha, coluna ou grade de 3×3 contenha células com listas possíveis de 467, 46 e 67. Claramente, uma dessas células precisa ser 4, uma precisa ser 6 e uma precisa ser 7. Assim, 4, 6 e 7 podem ser removidos de todas as outras listas possíveis nessa linha, coluna ou grade de 3×3 .

As triplas também podem ser ocultas. Suponha que uma linha, coluna ou grade de 3×3 contenha as listas possíveis 5789, 259 e 13789, e que nenhuma outra célula nessa linha, coluna ou grade de 3×3 mencione 5, 7 ou 9. Então, uma dessas células precisa ser 5, uma precisa ser 7 e uma precisa ser 9. Chamamos 579 de **tripla oculta**, e todos os possíveis diferentes de 5, 7 e 9 podem ser excluídos dessas três células (ou seja, 5789 torna-se 579, 259 torna-se 59 e 13789 torna-se 79), simplificando um pouco mais o quebra-cabeça.

	1	2	3	4	5	6	7	8	9
1					4 7	6			
2					—				
3					—				
4					1 4 5 6 7				
5					—				
6					4 7	6			
7					—				
8					—				
9					4 7	6			

Figura D.7 ■ Uso de triplas para simplificar um quebra-cabeça.

Outras estratégias de solução do Sudoku

Existem muitas outras estratégias para solucionar um jogo de Sudoku. Aqui estão dois dos muitos sites que recomendamos em nosso Sudoku Resource Center (<www.deitel.com/sudoku>), que o ajudarão a se aprofundar no assunto:

www.sudokuoftheday.com/pages/techniques-overview.php

www.angusj.com/sudoku/hints.php

D.4 Programação de soluções para o Sudoku

Nesta seção, sugerimos como programar solucionadores de Sudoku. Usamos diversas técnicas. Algumas podem parecer pouco inteligentes, mas se elas podem solucionar Sudokus mais rapidamente que qualquer ser humano, então talvez elas sejam, de certa forma, inteligentes.

Se você resolveu nossos exercícios do Passeio do Cavalo (exercícios 6.24, 6.25 e 6.29) e os das Oito Rainhas (exercícios 6.26 e 6.27), já terá executado diversas técnicas de solução de problemas pela força bruta e pela heurística. Nas próximas seções, sugerimos estratégias de solução do Sudoku por força bruta e heurística. Você deverá tentar programá-las, além de criar e programar suas próprias estratégias. Nosso objetivo é, simplesmente, deixá-lo acostumado ao Sudoku, e a alguns de seus desafios e estratégias de solução de problema. Enquanto isso, você ficará mais confortável com a manipulação de arrays bidimensionais e com estruturas de iteração aninhadas. Não tentamos produzir estratégias ideais, de modo que, quando você analisá-las, pensará em como poderá melhorá-las.

Programação de uma solução para Sudokus ‘fáceis’

As estratégias que mostramos — eliminar possibilidades com base nos valores já confirmados na linha, coluna e grade de 3×3 de uma célula, e simplificar um quebra-cabeça usando singletons, duplas (e duplas ocultas) e triplas (e triplas ocultas) —, às vezes, são suficientes para solucionar um quebra-cabeça. Você pode programar as estratégias e depois repeti-las até que todos os 81 quadrados sejam preenchidos. Para confirmar que o quebra-cabeça preenchido é um Sudoku válido, você pode escrever uma função para verificar se cada linha, coluna e grade de 3×3 contém os dígitos de 1 a 9 uma única vez. Seu programa deverá aplicar as estratégias em ordem. Cada uma delas força um dígito em uma célula, ou simplifica um pouco o quebra-cabeça. Assim que qualquer uma das estratégias funcione, retorne ao início de seu loop e reaplique as estratégias em ordem. Quando uma estratégia não funcionar, experimente uma outra. Para Sudokus ‘fáceis’, essas técnicas devem gerar uma solução.

Programação de uma solução para Sudokus mais difíceis

Para Sudokus mais difíceis, seu programa eventualmente alcançará um ponto onde ainda terá células não confirmadas com listas de possibilidades, e nenhuma das estratégias simples que discutimos funcionará. Se isso acontecer, primeiro salve o estado do tabuleiro, e então gere o próximo movimento escolhendo aleatoriamente um dos valores possíveis para qualquer uma das células restantes. Depois, reavalie o tabuleiro, enumerando as possibilidades restantes para cada célula. Depois, experimente as estratégias básicas novamente, repetindo-as até que ou o Sudoku seja resolvido ou até que as estratégias parem de funcionar, ponto em que você poderá testar outra jogada aleatória novamente. Se você chegar a um ponto em que ainda existam células vazias, mas nenhum dígito possível para pelo menos uma dessas células, o programa deverá abandonar essa tentativa, restaurar o estado do tabuleiro que você salvou e iniciar a técnica aleatória novamente. Continue examinando até que uma solução seja encontrada.

D.5 Criação de novos quebra-cabeças de Sudoku

Em primeiro lugar, consideremos as técnicas de criação de Sudokus de 9×9 prontos e válidos, com os 81 quadrados preenchidos. Então, iremos sugerir como esvaziar algumas das células para criar quebra-cabeças que as pessoas possam tentar resolver.

Métodos de força bruta

Quando os computadores pessoais apareceram no final da década de 1970, eles processavam dezenas de milhares de instruções por segundo. Os computadores desktop de hoje normalmente processam *bilhões* de instruções por segundo, e os supercomputadores mais velozes do mundo podem processar *trilhões* de instruções por segundo! As técnicas de força bruta que poderiam ter exigido meses de computação na década de 1970 agora podem produzir soluções em segundos! Isso encoraja as pessoas que precisam de resultados rapidamente a programar técnicas de força bruta simples, além da obtenção de soluções mais imediatas do que se fosse preciso gastar tempo no desenvolvimento de estratégias de solução de problema ‘inteligentes’ e mais sofisticadas. Embora nossas técnicas de força bruta possam parecer pesadas, elas gerarão soluções mecanicamente.

Para fazer uso dessas técnicas, você precisará de algumas funções utilitárias. Crie a função

```
int validSudoku( int sudokuBoard[ 10 ][ 10 ] );
```

que recebe um tabuleiro de Sudoku como um array bidimensional de inteiros (lembre-se de que estamos ignorando a linha 0 e a coluna 0). Essa função deverá retornar 1 se um tabuleiro completo for válido, 2 se um tabuleiro parcialmente completo for válido e 0 se o tabuleiro inteiro for inválido.

Uma técnica de força bruta completa

Uma das técnicas de força bruta consiste simplesmente em selecionar todas as substituições possíveis dos dígitos de 1 a 9 em cada célula. Isso poderia ser feito com 81 estruturas for aninhadas, cada uma realizando um loop de 1 a 9. O número de possibilidades (9^{81}) é tão grande que você poderia dizer que não vale a pena tentar. Mas a vantagem dessa técnica é que, eventualmente, você terá encontrado *todas* as soluções possíveis, algumas das quais podendo aparecer mais cedo, por sorte.

Uma versão ligeiramente mais inteligente dessa técnica de força bruta seria verificar cada dígito a ser posicionado para ver se ele deixa o tabuleiro em um estado válido. Se isso acontecer, então prossiga colocando um dígito na próxima célula. Se o dígito que você estiver tentando colocar deixar o tabuleiro em um estado inválido, experimente todos os outros oito dígitos, em ordem. Se um deles funcionar, então prossiga para a célula seguinte. Se nenhum deles funcionar, então retorne à célula anterior e experimente seu próximo valor. As estruturas for aninhadas podem tratar disso automaticamente.

Técnica de força bruta com permutações de linha selecionadas aleatoriamente

Cada linha, coluna e grade de 3×3 em um Sudoku válido contêm uma permutação dos dígitos de 1 a 9. Existem $9!$ Ou seja, $9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 362.880$ dessas permutações. Escreva uma função

```
void permutations( int sudokuBoard[ 10 ][ 10 ] );
```

que receba um array bidimensional de 10×10 , e na parte de 9×9 dele, que corresponde a uma grade de Sudoku, preencha cada uma das nove linhas com uma permutação selecionada aleatoriamente dos dígitos de 1 a 9.

Este é um modo de gerar uma permutação aleatória dos dígitos de 1 a 9: para o primeiro dígito, simplesmente escolha um dígito aleatório de 1 a 9; para o segundo dígito, use um loop para gerar repetidamente um dígito aleatório de 1 a 9 até que um dígito *diferente* do primeiro seja selecionado; para o terceiro dígito, use um loop para gerar repetidamente um dígito aleatório de 1 a 9 até que um dígito diferente dos dois primeiros seja selecionado; e assim por diante.

Depois de colocar, aleatoriamente, nove permutações selecionadas nas nove linhas de seu array Sudoku, execute a função `validSudoku` no array. Se ela retornar 1, você terá terminado. Se retornar 0, basta fazer o loop novamente, gerar outras nove permutações aleatoriamente selecionadas dos dígitos de 1 a 9 nas nove linhas sucessivas do array Sudoku. O processo simples gerará Sudokus válidos. A propósito, essa técnica garante que todas as linhas sejam permutações válidas dos dígitos de 1 a 9, de modo que você deve incluir uma opção à sua função `validSudoku` que a faça verificar somente as colunas e as grades de 3×3 .

Estratégias de solução heurísticas

Quando estudamos o Passeio do Cavalo nos exercícios 6.24, 6.25 e 6.29, desenvolvemos uma heurística do tipo ‘esperar antes de tomar uma decisão’. Para relembrar, uma heurística é uma diretriz. Ela ‘soa satisfatória’, e parece ser uma regra razoável a seguir. Ela é programável, de modo que nos dá um meio de direcionar um computador para tentar resolver um problema. Mas as técnicas heurísticas não garantem o sucesso, necessariamente. Para problemas complexos como a solução de um quebra-cabeça de Sudoku, o número de posicionamentos possíveis dos dígitos 1-9 é enorme, de modo que o que esperamos ao usar uma heurística razoável é que ela evite desperdiçar tempo com possibilidades inúteis e, em vez disso, focalize em tentativas de solução que, muito mais provavelmente, gerarão sucesso.

Uma heurística de solução do Sudoku do tipo ‘esperar antes de tomar uma decisão’

Desenvolvamos uma heurística do tipo ‘esperar antes de tomar uma decisão’ para solucionar Sudokus. Em qualquer ponto na solução do Sudoku, poderemos categorizar o tabuleiro listando em cada célula vazia os dígitos de 1 a 9 que ainda sejam possibilidades em aberto para essa célula. Por exemplo, se uma célula contém 3578, então a célula eventualmente terá de se tornar 3, 5, 7 ou 8. Ao tentar solucionar um Sudoku, alcançamos um beco sem saída quando o número de dígitos possíveis a serem colocados em uma célula vazia se torna zero. Assim, considere a seguinte estratégia:

1. Associe a cada quadrado vazio uma lista de possibilidades com os dígitos que ainda podem ser colocados nesse quadrado.
2. Caracterize o estado do tabuleiro simplesmente contando o número de possíveis posicionamentos para o tabuleiro inteiro.
3. Para cada posicionamento possível em cada célula vazia, associe o contador que caracterizaria o estado do tabuleiro após esse posicionamento.
4. Então, coloque o dígito específico no quadrado vazio específico (de todos os restantes) que deixe a contagem do tabuleiro mais alta (no caso de um empate, escolha aleatoriamente). Esta é uma das chaves para ‘esperar antes de tomar uma decisão’.

Heurística da antecipação

Isso é simplesmente um adorno para a heurística de ‘manter suas opções abertas’. No caso de um empate, antecipe mais uma colocação. Coloque o dígito específico no quadrado específico cujo posicionamento subsequente deixe a contagem do tabuleiro mais alta após dois movimentos.

Criação de um Sudoku com células vazias

Quando seu programa gerador de Sudoku estiver funcionando, você deverá ser capaz de gerar muitos Sudokus válidos rapidamente. Para formar um quebra-cabeça, salve a grade resolvida e, depois, esvazie algumas células. Um modo de fazer isso é esvaziar células escolhidas aleatoriamente. Uma observação geral é que os Sudokus tendem a se tornar mais difíceis à medida que as células vazias aumentam (existem exceções para isso).

Outra técnica é esvaziar as células de modo a deixar o tabuleiro resultante simétrico. Isso pode ser feito programaticamente, ao se escolher aleatoriamente uma célula a ser desocupada, e então esvaziar sua ‘célula reflexiva’. Por exemplo, se você esvaziar a célula superior esquerda $s[1][1]$, poderia esvaziar a célula inferior esquerda $s[9][1]$ também. Essas reflexividades são calculadas ao se apresentar a coluna, mas determinando a linha ao se subtrair a linha inicial de 10. Você também poderia fazer as reflexividades subtraindo de 10 a linha e a coluna da célula que você está esvaziando. Logo, a célula reflexiva para $s[1][1]$ seria $s[10-1][10-1]$ ou $s[9][9]$.

Um desafio de programação

Os Sudokus publicados em geral têm exatamente uma solução, mas ainda é satisfatório solucionar qualquer Sudoku, até mesmo aqueles que possuem várias soluções. Desenvolva um meio de demonstrar que um jogo de Sudoku específico tem *exatamente uma* solução.

D.6 Conclusão

Este apêndice sobre solução e programação de Sudoku apresentou muitos desafios. Não deixe de verificar nosso Sudoku Resource Center (www.deitel.com/sudoku/) em busca de diversos recursos na Web que o ajudarão a dominar o Sudoku e a desenvolver várias técnicas para a escrita de programas que criam e solucionam os jogos de Sudoku existentes.